TRƯỜNG ĐẠI HỌC
SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH
HCMC University of Technology and Education
HCMUTE

KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN CÔNG NGHỆ PHẦN MỀM

CÔNG NGHỆ PHẦN MỀM MỚI (MTSE431179)

useEffect trong ReactHooks

THS. NGUYỄN HỮU TRUNG

Thông tin Giảng viên

- Ths. Nguyễn Hữu Trung
- Khoa Công Nghệ Thông Tin
- Trường Đại học Sư Phạm Kỹ Thuật TP.HCM
- 090.861.7108
- trungnh@hcmute.edu.vn
- https://www.youtube.com/@baigiai

Nội dung

- Giới thiệu về React Hook
- useEffect() là gì?
- Cú pháp và một số ví dụ về useEffect trong React
- Cách sử dụng useEffect() trong nhiều trường hợp?

Giới thiệu về React Hook

- React Hook là một tính năng được giới thiệu từ phiên bản React 16.8, giúp các lập trình viên dễ dàng quản lý **trạng thái** và **logic** của ứng dụng React.
- React Hook được thiết kế để thay thế hoàn toàn cho việc sử dụng **class components** trong React. Nó cho phép lập trình viên sử dụng **state** và **lifecycle methods** của React trong các **function components**. Nhờ đó, chúng ta có thể viết các thành phần dễ dàng hơn, không cần phải sử dụng **class components**, điều này giúp giảm đáng kể lượng code cần phải viết.
- React Hook bao gồm các hook được xây dựng sẵn, như useState, useEffect, useContext, useReducer, useMemo, useCallback và useRef. Mỗi hook có một chức năng riêng và được sử dụng trong các trường hợp khác nhau.
- Ví dụ, useState hook được sử dụng để quản lý trạng thái của một component, useEffect được sử dụng để quản lý side effect của một component, useContext được sử dụng để truy cập các giá trị context, và useRef được sử dụng để lưu trữ các tham chiếu tới các elements trong DOM.
- Việc sử dụng React Hook có nhiều lợi ích như giúp code dễ đọc hơn, dễ bảo trì hơn và giúp tăng hiệu suất của ứng dụng React. Với React Hook, chúng ta có thể tạo ra các thành phần dễ dàng hơn, không cần phải sử dụng class components, điều này giúp giảm đáng kể lượng code cần phải viết.

useEffect() ?

- **useEffect()** là một hook trong React giúp thực hiện các tác vụ gắn kết (side effects) như gọi API, thay đổi DOM, tương tác với các thư viện bên ngoài, và các tác vụ khác mà ảnh hưởng đến state của ứng dụng.
- Được đặt tên theo "Effect" trong **React**, **useEffect()** được sử dụng để thực hiện các tác vụ gắn kết của component React. Nó thường được sử dụng để gọi các hàm không thuộc về React nhưng cần được thực hiện khi state của component thay đổi.
- Cú pháp của useEffect() là:

```
1  useEffect(() => {
2    // thực hiện tác vụ gắn kết
3    return () => {
4      // thực hiện tác vụ hủy gắn kết
5    };
6  }, [dependencies]);
```

- Trong đó, callback function đầu tiên sẽ được thực thi khi component được render. Nếu muốn hủy các tác vụ được gắn kết khi component unmount, ta có thể trả về một hàm từ callback function đó.
- Dependency array, nếu được cung cấp, sẽ được sử dụng để giới hạn việc gọi callback function chỉ khi các dependency được liệt kê trong array thay đổi.

useEffect() ?

- Ví dụ, ta có thể sử dụng useEffect() để gọi API bằng axios và lưu trữ dữ liệu vào state của component như sau:

```
import { useState, useEffect } from 'react';
function App() {
  const [data, setData] = useState([]);
  useEffect(() => {
    fetch('http://iotstar.vn/v1/api/posts')
      .then(response => response.json())
      .then(<a href="#">json</a> => setData(json));
  }, []);

  return (
    <div>
      {data.map(item => (
        <p key={item.id}>{item.title}</p>
      ))}
    </div>
  );
}
```

- Trong ví dụ này, ta sử dụng useEffect() để gọi API và lưu trữ dữ liệu trả về vào state của component.
- Hàm callback trong useEffect() chỉ được gọi một lần khi component được render. Dependency array được cung cấp là một mảng rỗng, điều này đảm bảo rằng hàm callback chỉ được gọi một lần khi component được render lần đầu tiên

useEffect() ?

- Cách sử dụng useEffect() trong nhiều trường hợp:
  - Tải dữ liệu từ API: Khi một component được render, có thể sử dụng useEffect để gọi API để lấy dữ liệu. useEffect sẽ được gọi mỗi khi có sự thay đổi trong dependency array

```
import { useState, useEffect } from 'react';
function App() {
  const [data, setData] = useState([]);
  useEffect(() => {
    fetch('http://iotstar.vn/v1/api/posts')
      .then(response => response.json())
      .then(data => setData(json));
  }, []);
  return (
    <div>
      {data.map(item => (
        <p key={item.id}>{item.title}</p>
      ))}
    </div>
  );
}
```

useEffect() ?

- Cách sử dụng useEffect() trong nhiều trường hợp:
  - Tương tác với DOM: Khi cần thực hiện các thao tác với DOM, chẳng hạn như thêm hoặc xoá các element HTML, có thể sử dụng useEffect với các tham số dependency để trigger sự thay đổi

```
function MyComponent() {
  const [visible, setVisible] = useState(false);

  useEffect(() => {
    const element = document.getElementById('my-element');
    if (visible) {
      element.classList.add('visible');
    } else {
      element.classList.remove('visible');
    }
  }, [visible]);

  return (
    <div>
      <button onClick={() => setVisible(true)}>Show element</button>
      <button onClick={() => setVisible(false)}>Hide element</button>
      <div id="my-element"><a href="#">This</a> is my element</div>
    </div>
  );
}
```

useEffect() ?

- Cách sử dụng useEffect() trong nhiều trường hợp:
  - Subscribing và unsubscribing event listener: Khi tạo event listener, useEffect được sử dụng để đăng ký và hủy đăng ký listener. Điều này giúp tránh tình trạng "leaking" event listener trong các component không còn được sử dụng

```
function MyComponent() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    const handleClick = () => setCount(count + 1);
    document.addEventListener('click', handleClick);
    return () => {
      document.removeEventListener('click', handleClick);
    };
  }, [count]);

  return (
    <div>
      <p>Count: {count}</p>
    </div>
  );
}
```

useEffect() ?

- Cách sử dụng useEffect() trong nhiều trường hợp:
  - Subscribing và unsubscribing event listener: Khi tạo event listener, useEffect được sử dụng để đăng ký và hủy đăng ký listener. Điều này giúp tránh tình trạng "leaking" event listener trong các component không còn được sử dụng

```
function MyComponent() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    const handleClick = () => setCount(count + 1);
    document.addEventListener('click', handleClick);
    return () => {
      document.removeEventListener('click', handleClick);
    };
  }, [count]);

  return (
    <div>
      <p>Count: {count}</p>
    </div>
  );
}
```

https://legacy.reactjs.org/docs/hooks-effect.html

CÔNG NGHỆ PHẦN MỀM MỚI (MTSE431179)

useContext trong React

THS. NGUYỄN HỮU TRUNG

Nội dung

- Giới thiệu về useContext trong React Js
- Ví dụ về useContext

Giới thiệu về useContext

- useContext là một trong những hooks cơ bản của React, được sử dụng để truyền dữ liệu giữa các thành phần con trong cây thành phần của ứng dụng. Nó cho phép tránh việc truyền các props qua nhiều cấp thành phần, giúp làm cho mã nguồn của bạn trở nên sáng sủa hơn.
- Sử dụng useContext, chúng ta có thể truy cập trực tiếp vào bất kỳ đối tượng Context nào trong cây thành phần của ứng dụng, và lấy các giá trị mà nó cung cấp cho các thành phần con.
- **Cách setup useContext trong React**
  - Để sử dụng useContext, trước hết bạn cần tạo một đối tượng Context để chứa các giá trị bạn muốn chia sẻ cho các thành phần con. Để làm điều này, bạn có thể sử dụng hàm createContext() của React như sau:

```
import React, { useContext } from 'react';
import MyContext from './MyContext';

<a href="#">function</a> MyComponent() {
  const contextValue = useContext(MyContext);
  // sử dụng giá trị trong contextValue ở đây
}
```

- Ở đây, useContext(MyContext) sẽ trả về giá trị hiện tại của Context được cung cấp bởi Provider bao thành phần hiện tại.

Ví dụ về useContext

- Trong ví dụ, chúng ta đã tạo một Context với tên là ThemeContext bằng cách sử dụng React.createContext. Sau đó, chúng ta đã cung cấp giá trị dark cho Context này thông qua việc sử dụng ThemeContext.Provider trong component App.
- Cuối cùng, chúng ta đã sử dụng useContext để lấy giá trị của Context đó và áp dụng nó vào một thuộc tính của button. Khi button được hiển thị, nó sẽ có background màu đen, tương ứng với giá trị dark được cung cấp cho ThemeContext.Provider.

```
import React, { useContext } from "react";

const ThemeContext = React.createContext("light");

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Toolbar />
    </ThemeContext.Provider>
  );
}

function Toolbar() {
  const theme = useContext(ThemeContext);
  return (
    <div>
      <button style={{ background: theme }}>Click me</button>
    </div>
  );
}
```